

Production Docker Cheat Sheet

■ Container Management

`docker run -d -p 8080:80 --name web nginx` → Run a detached container with port mapping
`docker run -it ubuntu bash` → Run an interactive container terminal shell
`docker ps` → List active containers
`docker ps -a` → List all active and stopped containers
`docker logs <container_id>` → Stream container runtime logs
`docker exec -it <container_id> sh` → Open an interactive terminal execution shell inside a live container
`docker stop <container_id>` → Safely stop a container
`docker rm <container_id>` → Permanently remove a container

■ Image Operations

`docker build -t app:v1.0 .` → Build an image from a local Dockerfile
`docker images / docker image ls` → List all locally cached images
`docker tag app:v1.0 username/app:v1.0` → Retag an image to prepare for shipping
`docker push username/app:v1.0` → Publish an image to Docker Hub
`docker rmi <image_id>` → Remove a local image layer

■ Storage Volumes

`docker volume create my-data` → Provision a new isolated storage disk
`docker volume ls` → List available volume systems
`docker volume inspect my-data` → View disk mount details and host storage paths
`docker volume rm my-data` → Delete a specific volume drive

■ Network Bridges

`docker network create app-net` → Initialize a custom isolated network bridge
`docker network ls` → List all internal Docker networks
`docker network inspect app-net` → View containers connected to a network bridge
`docker network connect app-net web-app` → Connect a live container to a network

■ Docker Compose

`docker compose up -d` → Build, network, and spin up an entire stack in the background
`docker compose down` → Stop and destroy the service stack
`docker compose down -v` → Destroy the stack and associated volumes
`docker compose ps` → Show service status
`docker compose logs -f` → Follow logs of the stack

■ Infrastructure Cleanup

`docker system df` → Analyze Docker disk usage
`docker system prune -a --volumes` → Remove stopped containers, unused networks, caches, and unattached volumes

■ Dockerfile Reference Blueprint

`FROM` → Base image
`WORKDIR` → Working directory
`COPY` → Copy project files
`RUN` → Execute build commands
`ENV` → Environment variables
`EXPOSE` → Document open ports
`CMD` → Default startup command